TRNGs para generación de secuencias muy largas

por

Ciro Fabián Bermúdez Márquez

Tesis presentada en cumplimiento parcial de los requisitos para el grado de:

MSc. en Electrónica

en

Instituto Nacional de Astrofísica, Óptica y Electrónica (INAOE)

Julio, 2021

Santa María de Tonantzintla, Puebla

Asesores:

Dr. Esteban Tlelo Cuautle

Departamento de Electrónica INAOE

Dr. Cuauhtemoc Mancillas López

Departamento de Electrónica INAOE

©INAOE 2021

Todos los derechos reservados

El autor otorga a INAOE el permiso para reproducir y distribuir este documento.

Agradecimientos

Aquí van unos agradecimientos bien locochones.

Índice general

Agradecimientos I

Índice general IV

Índice de figuras V

Índice de tablas VII

Lista de códigos IX

Lista de símbolos XI

Resumen XIII

1. Introducción 1

1.1. Estado del arte 1

1.1.1. Presentación del tema 1

1.1.2. Revisión teórica metodológica 14

1.1.3. Aplicaciones 14

1.1.4. Perspectivas de desarrollo 14

1.2. Objetivos 14

1.2.1. Objetivo general 14

1.2.2. Objetivos específicos 14

2. Selección y evaluación de núcleos TRNGs 15

2.1. Arquitectura de núcleos TRNG seleccionados 15

2.1.1. Elementary ring oscillator based TRNG (ERO-TRNG) 15

2.1.2. Coherent sampling ring oscillator based TRNG (COSO-TRNG) 15

2.1.3. Multi-ring oscillator based TRNG (MURO-TRNG) 17

2.1.4. Transient effect ring oscillator based TRNG (TERO-TRNG) . . 17

2.1.5. Self-timed ring based TRNG (STR-TRNG) 17

2.1.6. PLL based TRNG (PLL-TRNG) 17

3. AES / algoritmo Rijndael **19**
3.1. Evolución de los códigos y su impacto en la historia 19
3.2. Importancia de los códigos hoy 19
3.3. Conceptos importantes 23

4. Pruebas NIST (National Institute of Standards and Technology) **25**
4.1. Introducción a las pruebas de números aleatorios 25
4.1.1. Aleatoriedad 26
4.1.2. Imprevisibilidad 26
4.1.3. Generadores de números aleatorios (RNG) 27
4.1.4. Generadores de números pseudoaleatorios (PRNG) 28
4.1.5. Testing (Pruebas) 28

5. Conclusiones **33**
5.1. Fichas bibliográficas 33

A. Códigos **35**

Bibliografía **37**

Índice de figuras

1.1. TRNG estructura general 4

2.1. Arquitectura del núcleo ERO-TRNG. 16
2.2. Arquitectura del núcleo COSO-TRNG. 16
2.3. Arquitectura del núcleo MURO-TRNG. 16
2.4. Arquitectura del núcleo TERO-TRNG. 17
2.5. Arquitectura del núcleo STR-TRNG. 17
2.6. Arquitectura del núcleo PLL-TRNG. 18

Índice de tablas

2.1. Resumen de los resultados de implementación de las TRNGs 18

4.1. Tipos de errores. 29

Lista de códigos

A.1. Nombre de abajo 35
A.2. Nombre de abajo 35

Lista de símbolos

COSO-TRNG	Coherent sampling ring oscillator based TRNG
ERO-TRNG	Elementary ring oscillator based TRNG
FPGA	Field Programmable Logic Array
MURO-TRNG	Multi-ring oscillator based TRNG
PLL-TRNG	Phase-locked loop based TRNG
RNG	Random Number Generator
STR-TRNG	Self-timed ring based TRNG
TERO	Transient Effect Ring Oscillator
TERO-TRNG	Transient effect ring oscillator based TRNG
TRNG	True Random Number Generator

Resumen

Aquí va un resumen bien padre de toda la tesis.

Capítulo 1

Introducción

Los números aleatorios se utilizan en muchas áreas de nuestras vidas

1.1. Estado del arte

1.1.1. Presentación del tema

Antecedentes

Una buena criptografía requiere buenos números aleatorios. La mayoría de protocolos criptográficos requieren la generación y el uso de valores secretos que deben ser desconocidos para los atacantes. Dado que los protocolos de seguridad se basan en la imprevisibilidad de las claves que utilizan, los generadores de números aleatorios para aplicaciones criptográficas deben cumplir requisitos estrictos. [5]

En 1999, Intel introdujo un generador de números aleatorios (RNG) basado en silicio que se incorporó en sus placas base, conjuntos de chips y sistemas de escritorio. Ese RNG fue el primero de la familia de primitivos de Intel, lanzado para la protección de datos y comunicaciones dentro del hardware de la PC. Intel sugirió que para definir el concepto de RNG, primero es necesario comprender la idea de aleatoriedad, que generalmente se asocia con la imprevisibilidad, el azar y la suerte. El área de las matemáticas proporciona una definición precisa de aleatoriedad que luego se puede aplicar a la evaluación de un RNG. [5] [2]

La mayoría de las fuentes de números aleatorios en realidad utilizan un generador pseudoaleatorio (PRNG). Los PRNG utilizan procesos deterministas para generar una serie de resultados a partir de un estado semilla inicial. Debido a que la salida es puramente una función de los datos semilla, la entropía real de la salida nunca puede exceder la entropía de la semilla. Es fundamental que un PRNG se genere correctamente desde una fuente confiable [5]

Un verdadero generador de números aleatorios (TRNG) utiliza una fuente no de-

terminista para producir aleatoriedad. La mayoría opera midiendo procesos naturales impredecibles, como el ruido térmico, el ruido atmosférico o la descomposición nuclear. Un PRNG por sí mismo será inseguro sin un TRNG para generar una semilla. Generar una semilla requiere una fuente de verdadera aleatoriedad, ya que es imposible crear verdadera aleatoriedad dentro de un sistema determinista.[5]

Los números aleatorios se pueden definir como una secuencia de números independientes con una distribución específica y una probabilidad específica de caer en cualquier rango de valores dado. Como resultado, el RNG ideal proporcionará un flujo de bits independientes, no deterministas y uniformemente distribuidos en un conjunto de datos infinito. Además, debido a que la evaluación matemática de la aleatoriedad es difícil, solo es posible usar análisis estadísticos en conjuntos de datos de muestra para detectar características que apunten a la no aleatoriedad, y una forma es realizando pruebas del NIST. suite [2]

En enero de 1997, la US National Institute of Standards and Technology (NIST) anunció el inicio de una iniciativa para desarrollar un nuevo estándar de encriptación: el AES. El nuevo estándar de cifrado se convertiría en un estándar federal de procesamiento de información (FIPS), reemplazando al antiguo estándar de cifrado de datos (DES) y Triple DES. [17]

Definiciones

Los generadores de números aleatorios o Random number generators (RNGs) por siglas en inglés son una de las primitivas criptográficas básicas utilizadas para diseñar protocolos criptográficos. Sus aplicaciones incluyen, pero no se limitan a, la generación de claves criptográficas, vectores de inicialización, desafíos, nonces y valores de relleno, y la implementación de contramedidas contra ataques de canal lateral. Los RNGs destinados a aplicaciones criptográficas deben cumplir requisitos básicos de seguridad. En primer lugar, sus valores de salida deben tener buenas propiedades estadísticas y ser impredecibles. En los diseños modernos, se requieren algunas características adicionales: el generador debe ser intrínsecamente seguro, robusto y resistente a los ataques y/o probado en línea mediante pruebas específicas del generador. [20]

Hay tres clases básicas de RNG utilizadas en criptografía:

1. Deterministic (pseudo-) random number generators (PRNG) son en su mayoría rápidos y tienen buenas propiedades estadísticas. Por lo general, se utilizan como generadores de claves en cifrados de flujo. Debido a la existencia de algunos algoritmos subyacentes, los PRNG son fáciles de implementar en dispositivos lógicos. Sin embargo, si se conoce el algoritmo, la salida del generador es predecible. Incluso cuando no se conoce el algoritmo pero se han grabado algunas de las secuencias de salida del generador, su comportamiento durante la secuencia

grabada puede utilizarse en futuros ataques. Por esta razón, los generadores de números pseudoaleatorios deben ser computacionalmente seguros (es decir, el algoritmo subyacente no se puede adivinar computacionalmente) y su valor inicial nunca debe reutilizarse. Las secuencias binarias (contadores) cifradas mediante un algoritmo de cifrado seguro se consideran seguras desde el punto de vista informático. La reutilización del valor inicial puede evitarse guardando el último valor del contador y utilizando el siguiente valor del contador la próxima vez.

2. Physical (true-) random number generators (TRNG) utilizan procesos físicos para generar números aleatorios. Si el proceso físico subyacente no se puede controlar, la salida del generador es impredecible y/o incontrolable. La velocidad final de los TRNG está limitada por el espectro de la señal aleatoria y por el principio utilizado para extraer la entropía de la misma (por ejemplo, frecuencia de muestreo vinculada con el espectro de ruido). Las características estadísticas de los TRNG están estrechamente relacionadas con la calidad de la fuente de entropía, pero también con el método de extracción de aleatoriedad. Debido a que los procesos físicos están sujetos a fluctuaciones, las características estadísticas de los TRNG suelen ser peores que las de los PRNG.

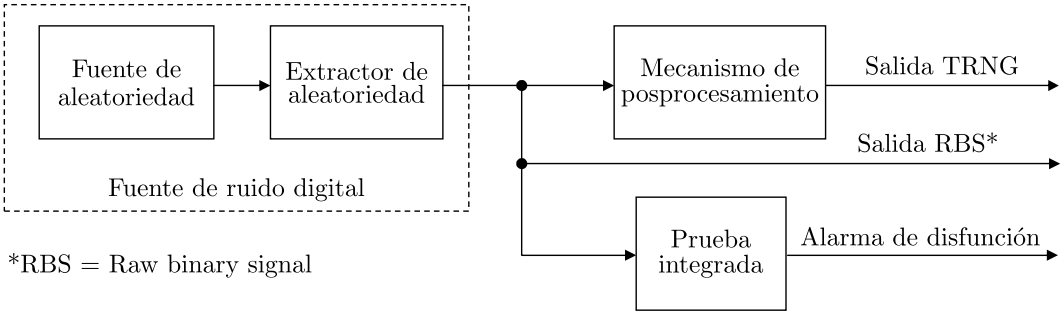
3. Hybrid random number generators (HRNG) presentan una combinación de un RNG determinista (rápido y de buena calidad) sembrado repetidamente por un RNG físico (lento pero impredecible). El diseñador debe encontrar un compromiso satisfactorio entre la velocidad del generador y su previsibilidad (ajustando el intervalo de tiempo entre las semillas y el tamaño de una semilla).

A pesar de la menor velocidad de los TRNG, se utilizan con más frecuencia en aplicaciones criptográficas que los PRNG. Es interesante notar que los TRNG son las únicas primitivas criptográficas que no han sido objeto de estandarización hasta el momento. Sin embargo, antes de usar el generador en la práctica, el principio y su implementación dentro de un módulo criptográfico deben ser validados por una institución acreditada como parte de un proceso de evaluación de seguridad. Los generadores que no cuentan con un certificado de seguridad se consideran inseguros en cuanto a su uso en aplicaciones criptográficas. Por ello, el estudio de los principales principios TRNG existentes y de sus características presentados en este capítulo es de gran interés.

La estructura general de un TRNG se muestra en la Figura 1.1. El generador debe utilizar un proceso físico incontrolable como fuente de aleatoriedad. Dado que los fenómenos físicos utilizados en los TRNG son en su mayoría procesos analógicos, suele ser necesario algún método que permita la conversión de datos del dominio analógico al digital. Se puede incluir una conversión de analógico a digital en el procedimiento de extracción de aleatoriedad. La señal binaria sin procesar obtenida (el llamado ruido

digital) puede tener baja entropía y/o malas propiedades estadísticas. En este caso, se pueden usar algunos algoritmos de procesamiento posterior para mejorar los parámetros estadísticos del flujo de bits de salida. Sin embargo, el procesamiento posterior de la salida TRNG a veces puede enmascarar una falla grave en el generador. Las pruebas estadísticas estándar pueden entonces fallar en detectar la debilidad enmascarada. Por lo tanto, se recomienda tener la posibilidad de probar el ruido digital sin procesar. La seguridad del generador se puede aumentar si las pruebas estadísticas se aplican sobre la marcha y si están diseñadas para adaptarse al principio del generador con particular referencia a sus posibles debilidades.

Figura 1.1: TRNG estructura general



Los verdaderos generadores de números aleatorios usan diferentes fuentes de aleatoriedad y numerosos principios para extraerla. A continuación, evaluamos los principios fundamentales de TRNG utilizando tres clases de características: parámetros relacionados con la calidad, parámetros relacionados con la seguridad y parámetros relacionados con el diseño.

- Parámetros relacionados con la calidad
  - Fuente de aleatoriedad
  - Método de extracción de aleatoriedad y entropía del ruido digital
  - Método de procesamiento posterior aplicado (opcional)
  - Tasa de bits de salida y su estabilidad
- Parámetros relacionados con la seguridad
  - Existencia de un modelo matemático
  - Comprobabilidad interna
  - Seguridad (robustez, resistencia contra ataques)
- Parámetros relacionados con el diseño
  - El uso de recursos

- El consumo de energía
- Viabilidad en dispositivos lógicos y FPGAs
- Automatización del diseño

Es importante señalar que estas características de los TRNG no son todas igualmente importantes. Los parámetros de seguridad como la robustez, la disponibilidad de un modelo estocástico, la capacidad de prueba, etc. siempre tienen prioridad en un sistema de seguridad de datos. Su peso en la evaluación de TRNG es mucho mayor que el de otros parámetros como el consumo de energía, la tasa de bits, etc.

- Fuentes de aleatoriedad en dispositivos lógicos

  Los dispositivos lógicos están diseñados para la implementación de sistemas lógicos deterministas. Cada comportamiento impredecible en un sistema de este tipo (causado por metaestabilidad, ()fluctuaciones de reloj (clock jitter), errores de radiación, etc.) puede tener consecuencias catastróficas para el comportamiento del sistema en general. Aunque los eventos impredecibles debido a la naturaleza física de la tecnología subyacente son inevitables, los proveedores de dispositivos lógicos tienden a minimizarlos. Por esta razón, los métodos de extracción de aleatoriedad utilizados en el diseño TRNG deben examinarse críticamente para mantenerse al día con la evolución de la tecnología subyacente. La mayoría de los dispositivos lógicos no contienen bloques analógicos, por lo que las fuentes de aleatoriedad están relacionadas con el funcionamiento de las puertas lógicas. Se pueden utilizar varios fenómenos y sus combinaciones: variación en el retardo de las puertas lógicas, comportamiento analógico de las puertas lógicas entre dos niveles lógicos (p. ej., metaestabilidad), violación del tiempo de establecimiento y retención y ruido térmico generado dentro del dispositivo.

  La inestabilidad de los retardos de las puertas lógicas provoca variaciones en la propagación de la señal a lo largo del tiempo. Estas variaciones pueden verse como una inestabilidad del período del reloj (la fluctuación o jitter) en los generadores de reloj que contienen elementos de retardo ensamblados en un circuito cerrado (osciladores en anillo). La variación en el tiempo de propagación también se utiliza en generadores con elementos de retardo en un conjunto de cadena abierta. La cadena se utiliza para aumentar o ajustar el retardo total.

  Dado que las resistencias y los condensadores se pueden implementar fácilmente en la tecnología digital, el ruido térmico generado en las resistencias se puede utilizar para modular la frecuencia de un oscilador de funcionamiento libre (oscilador RC). El ruido térmico se convierte así al dominio del tiempo, donde se puede extraer fácilmente. Sin embargo, este principio no se puede utilizar en FPGA, porque no se dispone de estructuras apropiadas.

Algunos generadores utilizan la fluctuación de seguimiento introducida por los bucles de bloqueo de fase o phase locked loops (PLL) para generar números aleatorios. Estos llamados PLL analógicos son fáciles de implementar en dispositivos digitales (incluidos los FPGA), porque el filtro RC que se encuentra en dichos PLL, es principalmente el único bloque "analógico", se puede implementar fácilmente usando la misma tecnología. Por lo tanto, podemos considerar un TRNG basado en PLL como un generador que puede implementarse en dispositivos lógicos en general.

- Métodos de extracción de aleatoriedad

  En los dispositivos lógicos que no contienen un bloque analógico, la aleatoriedad a menudo se extrae mediante el muestreo de una señal (de reloj) en los flancos ascendentes o descendentes de una señal de referencia (reloj) utilizando flip-flops (latches) síncronos o asíncronos. El flujo de bits aleatorio se puede obtener de dos formas: muestreando señales aleatorias a intervalos regulares o muestreando señales regulares a intervalos de tiempo aleatorios. En sistemas síncronos, el primer método es preferible para garantizar una tasa de bits constante en la salida.

  La elección entre flip-flops síncronos y asíncronos no parece ser importante en ASIC, pero sí lo es en FPGA. Esto se debe a que los flip-flops síncronos están cableados en celdas lógicas como bloques optimizados y, en consecuencia, su comportamiento metaestable se minimiza. Por otro lado, los latches generalmente solo se pueden implementar en Look up Tables (LUT) y, por lo tanto, están sujetos a un comportamiento metaestable en mayor medida. Hasta el momento, el comportamiento de flip-flops síncronos y asíncronos y su uso para la extracción de aleatoriedad en FPGAs no ha sido suficientemente evaluado.

  El método de extracción de aleatoriedad suele estar vinculado al principio básico del generador y a la fuente de aleatoriedad. El procedimiento de extracción de aleatoriedad y el posprocesamiento a veces se fusionan en el mismo bloque y no se pueden separar. En ese caso, la entropía de la fuente de aleatoriedad se modifica mediante el posprocesamiento y no se puede medir ni evaluar correctamente.

  En el verdadero proceso de evaluación del generador de números aleatorios, la fuente de aleatoriedad y el método de extracción de aleatoriedad están estrechamente vinculados y no se pueden disociar. Por esta razón, es más razonable evaluar estos dos parámetros del generador juntos. La mejor forma de hacerlo es evaluando la entropía incluida en el ruido digital.

- Posprocesamiento de la señal binaria sin procesar

  La evaluación de los TRNG casi siempre se basa en pruebas estadísticas que se aplican a secuencias aleatorias producidas por el TRNG. A veces, la fuente

de entropía puede tener algunas debilidades que conducen a la producción de números no aleatorios (secuencias largas de ceros o unos). Por esta razón, el posprocesamiento puede ser necesario para mejorar las propiedades estadísticas de los números aleatorios, por ejemplo, para aumentar la entropía por bit, reducir el sesgo y/o la correlación.

La calidad de la señal de ruido digital (la señal obtenida en el bloque de extracción de aleatoriedad) puede deteriorarse por varias razones: (a) la entropía de la fuente no es lo suficientemente alta (este suele ser el caso si se utiliza la metaestabilidad como fuente de aleatoriedad).); (b) la entropía, que es alta en la señal original, no está bien extraída; (c) las muestras extraídas están correlacionadas. La entropía por bit a la salida del generador aumenta principalmente a costa de la reducción y/o variación de la tasa de bits.

- Corrector XOR
- Corrector de Von Neumann
- Linear Feedback Shift Registers (LFSRs)
- Funciones resilientes
- Cifrado de la señal de ruido digital
- Hashing de la señal de ruido digital

- Tasa de bits de salida (Output Bit Rate)

  La velocidad es un parámetro secundario (después de la seguridad) en muchas aplicaciones criptográficas. Las velocidades de bits de salida desde cien kilobits por segundo hasta 1 megabit por segundo suelen ser suficientes. Sin embargo, hay algunas aplicaciones de seguridad de datos de velocidad crítica para las que se requieren generadores de alta velocidad. Por ejemplo, la criptografía cuántica requiere una tasa de bits alta (hasta 100 megabits por segundo) debido a la muy baja eficiencia de la transmisión de datos clave a través del canal óptico de baja potencia.

  Los servidores de telecomunicaciones de alta velocidad son un segundo ejemplo. Necesitan generar claves de sesión de forma regular a alta velocidad (decenas de megabits por segundo). Por ejemplo, un concentrador/servidor Ethernet de 10 Gbit necesitaría alrededor de 20 Mbits/s de bits aleatorios para generar una clave de sesión de 128 bits para cada bloque de datos de 64 kB para poder resistir los ataques de canal lateral.

  Otro aspecto de la tasa de bits de salida que debe tenerse en cuenta es su variabilidad. Algunos generadores dan números aleatorios periódicamente, otros generan

resultados en intervalos de tiempo irregulares. En el segundo caso, se requiere un FIFO (First In First Out) para acumular los números generados. Otra solución es estimar la tasa de bits más pequeña disponible en la salida y muestrear la salida a esta tasa. La desventaja de la primera solución es que, según la tasa de bits de salida media y la necesidad de números aleatorios, los FIFO a veces deben ser muy grandes. La desventaja de la segunda solución es que si la tasa de bits estimada es incorrecta, los números aleatorios a veces pueden no estar disponibles en la salida.

- Modelado de TRNG

  La parte principal de un proceso de certificación de seguridad se ocupa de la evaluación del diseño y la implementación del generador. El objetivo de la evaluación es cuantificar la entropía por bit aleatorio [34]. Sin embargo, la entropía es una propiedad de variables aleatorias y no de realizaciones observadas (números aleatorios). Para cuantificar la entropía, necesitamos analizar la distribución de las variables aleatorias, por ejemplo mediante el uso de un modelo estocástico. El modelo estocástico especifica una familia de distribuciones de probabilidad de variables aleatorias que permite verificar un límite de entropía inferior para la señal binaria sin procesar. En [34], los autores usan un modelo que da un límite inferior de la entropía condicional promedio por número aleatorio interno. El valor dado por el modelo se puede utilizar para probar la entropía de los números generados en tiempo real

- Testabilidad

  La capacidad de prueba interna significa que la estructura del generador permite la evaluación de la entropía de la señal binaria sin procesar (la señal binaria sin procesar debe estar disponible). Esta funcionalidad es requerida en los procedimientos de evaluación TRNG recientes [45]. Sin embargo, cuando la extracción de aleatoriedad y el procesamiento posterior se fusionan en el mismo proceso, la señal aleatoria sin procesar no está disponible. Incluso cuando esta señal está disponible, a veces se compone de un patrón pseudoaleatorio combinado con un flujo de bits verdaderamente aleatorio. El patrón pseudoaleatorio dificulta la evaluación estadística de la señal. Por esta razón, proponemos un nuevo nivel de comprobabilidad: una comprobabilidad interna absoluta. La salida del generador que presenta capacidad de prueba interna absoluta no incluye un patrón pseudoaleatorio y contiene solo un flujo de bits aleatorio verdadero. La salida de simulación funcional del generador con capacidad de prueba interna absoluta es siempre cero. Si (por alguna razón) la fuente de aleatoriedad falla en un generador de hardware, la salida de la prueba será cero. Este hecho se puede utilizar para

asegurar el generador.

- Evaluación de seguridad

  A menudo es muy difícil (ya veces imposible) construir un modelo estocástico para un generador en particular. En ese caso, se puede utilizar otro enfoque para validar el uso del generador en aplicaciones criptográficas. Este enfoque se basa en el análisis del impacto del entorno cambiante o un ataque al generador (evaluación de seguridad). Hay tres posibilidades: (i) existe prueba de que el generador no puede funcionar mal como resultado de un ataque o de un entorno cambiante, (ii) no existe prueba de seguridad ni ataque, (iii) se ha informado de algún ataque a un generador en particular.

- El uso de recursos

  Para evaluar varios principios de TRNG, es importante analizar los recursos necesarios para la implementación del hardware del generador. En términos generales, todo tipo de recursos disponibles en FPGA se pueden utilizar para generar números aleatorios: celdas lógicas basadas en LUT o basadas en multiplexor, bloques de memoria integrados, bloques de reloj con PLL y DLL, osciladores RC integrados, multiplicadores cableados, interconexiones programables, etc.

  Los FPGA tienen muchas celdas lógicas, por lo que el uso de celdas lógicas (el área lógica) no suele ser un problema. La topología y los parámetros eléctricos de las interconexiones programables dependen en gran medida de la tecnología. Muchos diseños de TRNG requieren la intervención manual del diseñador durante la colocación y el enrutamiento (P/R). Con respecto a P/R, algunos diseños pueden implementarse fácilmente en una familia de FPGA, pero son difíciles o imposibles de implementar en otras. La elección y el número de bloques cableados integrados suele ser mucho más limitado (PLL, osciladores RC, multiplicadores, bloques de memoria) y varía según el proveedor y la tecnología. Por lo tanto, el uso de bloques cableados puede ser un factor limitante para la reutilización del principio TRNG.

- El consumo de energía

  El consumo de energía del generador está vinculado a su fuente de aleatoriedad (por ejemplo, el oscilador), a la frecuencia de reloj utilizada y a la agilidad del algoritmo. En aplicaciones de energía crítica, el generador se puede detener cuando no está en uso. Sin embargo, la posibilidad de controlar la generación de flujo de bits se puede utilizar para atacar al generador.

- Viabilidad en FPGAs

En comparación con la implementación de TRNG en ASIC, su implementación en FPGA es mucho más restringida. Muchos TRNG implementados en ASIC utilizan componentes analógicos para generar aleatoriedad (p. ej., TRNG basados en el caos que utilizan convertidores de analógico a digital, generadores basados en osciladores de ejecución libre que utilizan ruido térmico de diodos y resistencias, etc.) y para procesar la aleatoriedad (p. ej., amplificadores operacionales, comparadores, etc. .). La mayoría de estos bloques funcionales generalmente no están disponibles en FPGA, aunque algunos de ellos pueden estar disponibles en familias seleccionadas, p. Osciladores RC en Actel Fusion FPGA, PLL analógicos en la mayoría de las familias Altera y Actel, pero no en las antiguas familias Xilinx. Desde el punto de vista de su factibilidad en FPGAs, algunos generadores no son factibles o son difíciles de implementar en FPGAs, algunos son factibles en FPGAs seleccionados y los principios más generales son factibles en todas las FPGAs.

- Automatización del diseño

  El hecho de que el generador utilice recursos que están disponibles en FPGAs no significa automáticamente que pueda ser implementado en este tipo de dispositivos. El rango de tolerancia de algunos parámetros tecnológicos puede ser tal que impida la implementación confiable del generador. El parámetro que más suele limitar la implementación del generador en FPGA es la disponibilidad de los recursos de enrutamiento y sus características. Algunos generadores requieren un enrutamiento perfectamente equilibrado. Esto requiere un control perfecto de la ubicación del módulo (por ejemplo, ubicación simétrica de dos módulos en relación con otro módulo) y enrutamiento. Si bien la mayoría de las herramientas de diseño de FPGA permiten un control preciso de la ubicación, el proceso de enrutamiento es difícil o imposible de controlar (por ejemplo, en la familia Actel). Incluso cuando el enrutamiento se puede controlar parcial o totalmente (por ejemplo, las familias Altera y Xilinx), los retrasos en la red de enrutamiento configurable varían tanto de un dispositivo a otro que es imposible equilibrar las interconexiones de los módulos de manera general y el diseño dependerá del dispositivo. , es decir, debe equilibrarse manualmente para cada dispositivo. Tal intervención manual no es aceptable desde el punto de vista de la implementación práctica del generador. Los mejores generadores (muy raros) se pueden mapear automáticamente (sin intervención manual) en todas las familias de FPGA. Desde el punto de vista práctico, sigue siendo aceptable si la implementación del generador requiere intervención manual (P/R) para cada familia y/o tipo de dispositivo. Sin embargo, los generadores que requieren optimización manual por dispositivo no son aceptables para aplicaciones industriales.

Cuestiones de calidad y seguridad en el diseño TRNG

La salida de un buen TRNG debería ser indistinguible de la salida de un TRNG ideal, independientemente de las condiciones y el tiempo de funcionamiento. La calidad del flujo de bits de salida del generador y sus parámetros de seguridad, incluida la robustez contra el envejecimiento, los cambios ambientales, los ataques, la existencia de autocomprobación y las pruebas en línea, son muy importantes en el diseño de TRNG.

Condiciones de funcionamiento y calidad de la salida TRNG

Se supone que los buenos TRNG proporcionan un flujo uniformemente distribuido de 0 y 1 en su salida. La calidad de la salida del generador está estrechamente ligada a la calidad de la fuente de aleatoriedad y al método de extracción de aleatoriedad utilizado. Las características espectrales de la fuente de aleatoriedad y la extracción de aleatoriedad determinan los parámetros principales del flujo de bits generado: el sesgo del flujo de bits de salida, la correlación entre los bits subsiguientes, los patrones visibles, etc. postprocesamiento, es mejor si el generador produce inherentemente un flujo de bits sin procesar de buena calidad.

Si el extractor muestrea la fuente de aleatoriedad demasiado rápido, los bits adyacentes podrían correlacionarse. Por esta razón, es una buena práctica verificar el flujo de bits generado para una correlación automática a corto plazo. También es posible que el ruido digital presente otras dependencias a corto plazo, que deben detectarse mediante algunas pruebas específicas del generador.

El comportamiento del generador a menudo está influenciado por interferencias eléctricas externas y/o internas. El efecto más obvio de esto serán las frecuencias discretas de la fuente de alimentación y de varias señales internas que aparecen en el espectro de ruido.

El espectro de la señal de ruido generada puede verse significativamente influenciado por un ruido 1/f de baja frecuencia causado por semiconductores. Además, las altas frecuencias del espectro de ruido pueden ser filtradas involuntariamente por algunas capacitancias internas.

Algunos generadores pueden presentar los llamados puntos defectuosos. Los puntos defectuosos son períodos breves en los que el generador deja de funcionar, posiblemente debido a interferencias eléctricas o a excursiones extremas de la parte sobrecargada del circuito del generador.

Otra característica peligrosa del generador puede ser una puerta trasera, que se refiere a las desviaciones de la aleatoriedad uniforme introducidas deliberadamente por el fabricante. Por ejemplo, supongamos que en lugar de usar algún proceso físico, el generador generaría una secuencia pseudoaleatoria de alta calidad con una semilla de 40 bits. Sería imposible detectar este comportamiento mediante la aplicación de pruebas estadísticas estándar en el flujo de bits de salida, pero sería computacionalmente factible

para alguien que conoce la puerta trasera adivinar claves sucesivas.

TRNG Evaluation and Testing

TRNG debe someterse al proceso de evaluación de calidad y seguridad antes de que pueda usarse en sistemas de seguridad de datos. Durante mucho tiempo, las pruebas TRNG se limitaron a evaluar la capacidad de las secuencias generadas para pasar una batería de pruebas estadísticas (FIPS 140-1 [17], NIST SP 800-22 [43] o DIEHARD [40, 41]).

Estrategia general de pruebas estadísticas

El papel de las pruebas estadísticas TRNG es evaluar la hipótesis de que el generador es realmente aleatorio. Esta hipótesis se llama hipótesis nula (H_0), porque se basa en nuestras creencias, en ausencia de evidencia. Las pruebas estadísticas deben proporcionar la evidencia de que la hipótesis es verdadera. Si las pruebas no tienen éxito, la hipótesis se rechaza, de lo contrario no lo es (pero no debe aceptarse por completo porque no hay evidencia de que la hipótesis sea cierta). Para cada prueba existente, se elige una estadística de aleatoriedad relevante y se utiliza para determinar el rechazo de la hipótesis nula. Bajo el supuesto de aleatoriedad, tal estadística tiene una distribución de valores posibles. Una distribución de referencia teórica de esta estadística bajo la hipótesis nula se determina mediante métodos matemáticos. A partir de esta distribución de referencia, se determina un valor crítico. Luego se calcula un valor estadístico en la secuencia y se compara con el valor crítico. Si el valor estadístico excede el valor crítico, se rechaza la hipótesis nula de aleatoriedad.

Un buen generador de números aleatorios verdaderos puede producir una secuencia que no parece ser aleatoria, mientras que un TRNG malo puede producir una secuencia aparentemente aleatoria. Normalmente se consideran dos tipos diferentes de errores:

- Tipo 1:
- Tipo 2:

Hay dos categorías principales de generadores de números aleatorios: generadores de números aleatorios deterministas (DRNG) y generadores de números aleatorios verdaderos (TRNG), que pueden ser físicos (P-TRNG) o no físicos (NP-TRNG). Mientras que los generadores deterministas se basan en procesos algorítmicos y, por lo tanto, no son verdaderamente aleatorios, los TRNG explotan un proceso impredecible, como los fenómenos analógicos en dispositivos electrónicos, para producir una secuencia binaria aleatoria o una secuencia de números aleatorios. La imprevisibilidad de los DRNG está garantizada computacionalmente y la de los TRNG está garantizada físicamente. Por lo tanto, es necesario un buen conocimiento del proceso físico subyacente a TRNG, que asegure su aleatoriedad y, por lo tanto, su imprevisibilidad. [19] [10] [12]

Las FPGAs o (Field Programmable Gate Arrays) por sus siglas en inglés son dispositivos semiconductores que se basan en una matriz de bloques lógicos configurables

(Configurable logic Blocks)(CLB) conectados mediante interconexiones programables. Los FPGA se pueden reprogramar para la aplicación deseada o los requisitos de funcionalidad después de la fabricación. [Xilinx Oficial]

Una FPGA es un circuito integrado semiconductor en el que se puede cambiar la mayor parte de la funcionalidad eléctrica dentro del dispositivo; cambiado por el ingeniero de diseño, cambiado durante el proceso de montaje de la placa de circuito impreso, o incluso cambiado después de que el equipo ha sido enviado a campo. Los FPGA brindan beneficios a los diseñadores de muchos tipos de equipos electrónicos, que van desde redes de energía inteligente, navegación de aviones, asistencia al conductor de automóviles, ultrasonidos médicos y motores de búsqueda de centros de datos, solo por nombrar algunos.[Intel Oficial]

Los osciladores de anillo o (Ring Oscillators) RO son estructuras combinaciones retroalimentadas basadas en inversores conectados en cadena y una compuerta AND. Los RO funcionan como fuentes de entropía física y debido a esto pueden ser usados dentro de las FPGAs para implementar TRNGs [14]

El TERO [8] [12] [19] [10]

AIS-20/31

Selección de núcleos para TRNG

Los TRNG destinados a ser utilizados en aplicaciones criptográficas deben cumplir con el estándar AIS-20/31. Dichos núcleos TRNG deben cumplir varios requisitos: 1) su diseño debe ser simple y comprensible, la fuente de aleatoriedad debe estar claramente definida; 2) el proceso aleatorio subyacente debe ser estacionario y el modelo estocástico debe ser factible; 3) la señal binaria sin procesar debe estar disponible para realizar más pruebas fuera de línea y en línea. Además, es útil si la fuente de aleatoriedad (p. ej., la fluctuación del reloj proveniente del ruido térmico) es cuantificable. Su medición dentro del dispositivo puede proporcionar una base para pruebas estadísticas integradas rápidas y eficientes.

Principios de Kerckhoffs

Áreas de abordaje

1.1.2. Revisión teórica metodológica

Descripción por área y comparación

1.1.3. Aplicaciones

1.1.4. Perspectivas de desarrollo

Áreas de trabajo actual

Problemas a resolver

Discusión

1.2. Objetivos

1.2.1. Objetivo general

- Objetivo general aquí

1.2.2. Objetivos específicos

1. Investigar los núcleos de TRNGs que pueden utilizarse en FPGAs.
2. Clasificación y especial atención a los TERO y DPLLs.
3. Implementación en FPGAs.
4. Clasificar los núcleos y
5. Hacer un post-procesamiento criptográfico utilizando AES.
6. Aprender a utilizar el system on chip de las FPGAs de Xilinx.
7. Como caso de estudio utilizaremos el TRNG para generar la semilla de un mapa caótico el cual será utilizado para generar una secuencia aleatoria muy larga, es decir, se utilizará como un generador pseudo aleatorio.
8. Finalmente con la conjunción tanto del TRNG como el mapa caótico se obtendrá un sistema completo para generar secuencias aleatorias de buena calidad que pueden utilizarse para cifrar y autenticar información

Capítulo 2

Selección y evaluación de núcleos TRNGs

2.1. Arquitectura de núcleos TRNG seleccionados

Con base en los criterios del AIS-20/31, y los núcleos TRNG que usan estructuras oscilantes como se clasifican en [8]:

- Single-event ring oscillators
  - Elementary ring oscillator based TRNG (ERO-TRNG)
  - Coherent sampling ring oscillator based TRNG (COSO-TRNG)
  - Multi-ring oscillator based TRNG (MURO-TRNG)
- Multi-event ring oscillators with signal collisions
  - Transient effect ring oscillator based TRNG (TERO-TRNG)
- Multi-event ring oscillators without signal collisions
  - Self-timed ring based TRNG (STR-TRNG)
- Phase-locked loops
  - PLL based TRNG (PLL-TRNG)

2.1.1. Elementary ring oscillator based TRNG (ERO-TRNG)

2.1.2. Coherent sampling ring oscillator based TRNG (COSO-TRNG)

Figura 2.1: Arquitectura del núcleo ERO-TRNG.

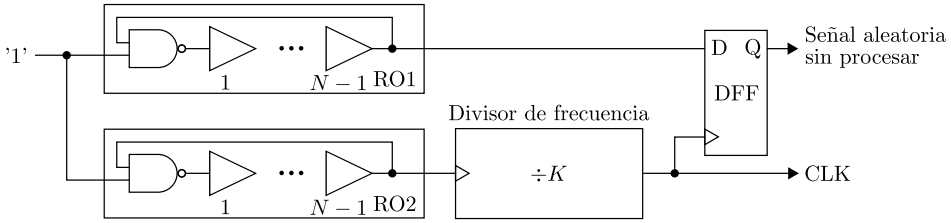


Figura 2.2: Arquitectura del núcleo COSO-TRNG.

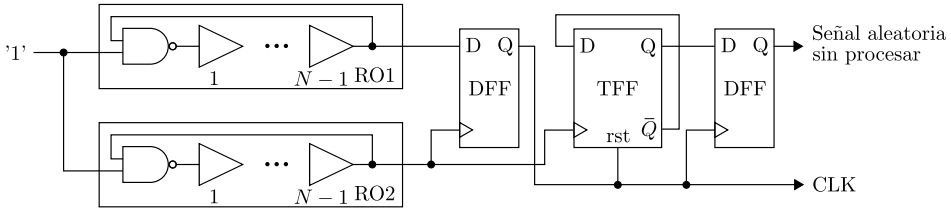
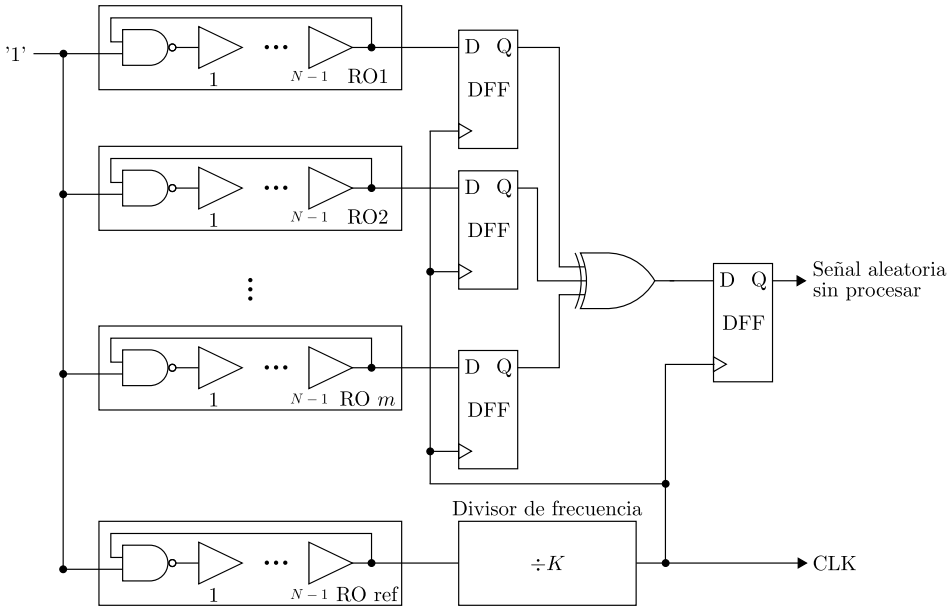


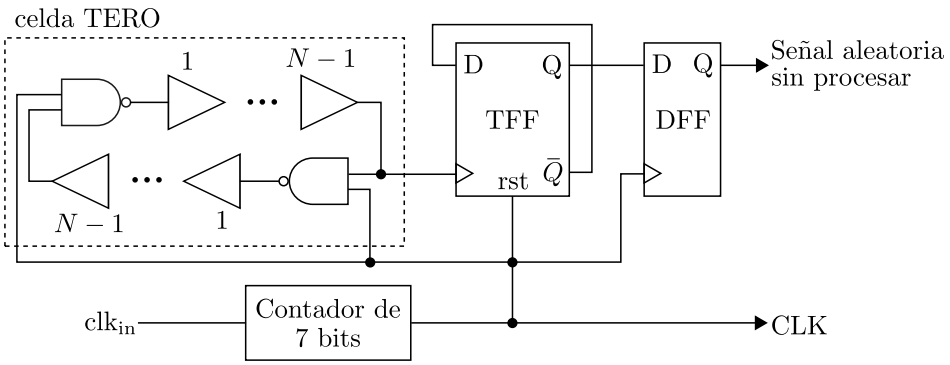
Figura 2.3: Arquitectura del núcleo MURO-TRNG.



2.1.3. Multi-ring oscillator based TRNG (MURO-TRNG)

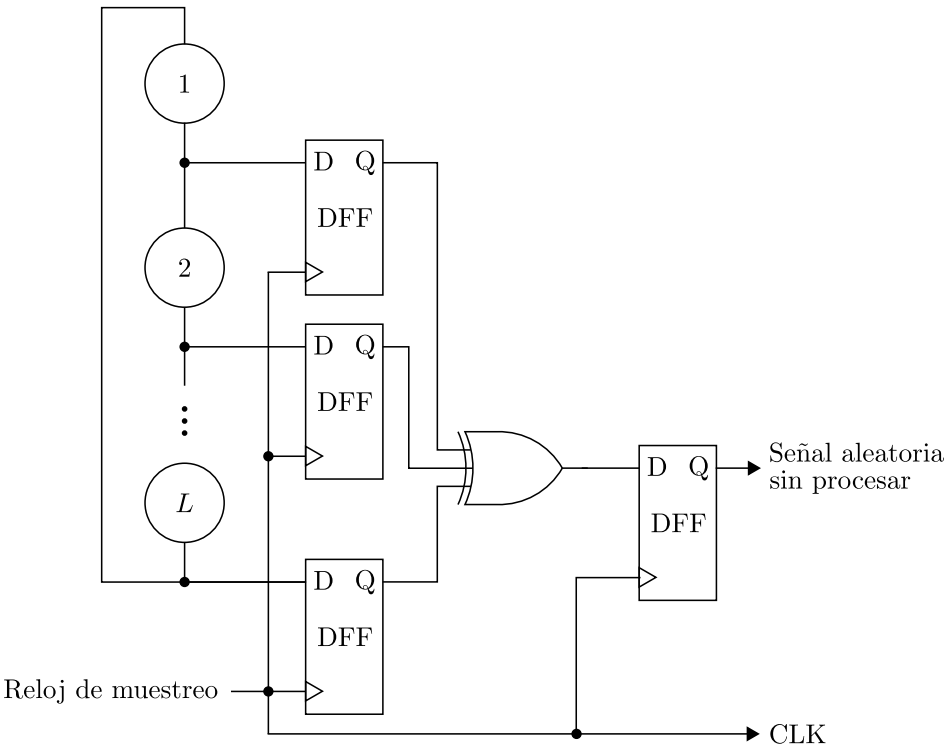
2.1.4. Transient effect ring oscillator based TRNG (TERO-TRNG)

Figura 2.4: Arquitectura del núcleo TERO-TRNG.



2.1.5. Self-timed ring based TRNG (STR-TRNG)

Figura 2.5: Arquitectura del núcleo STR-TRNG.



2.1.6. PLL based TRNG (PLL-TRNG)

FPGA.

RNG.

TERO.

Figura 2.6: Arquitectura del núcleo PLL-TRNG.

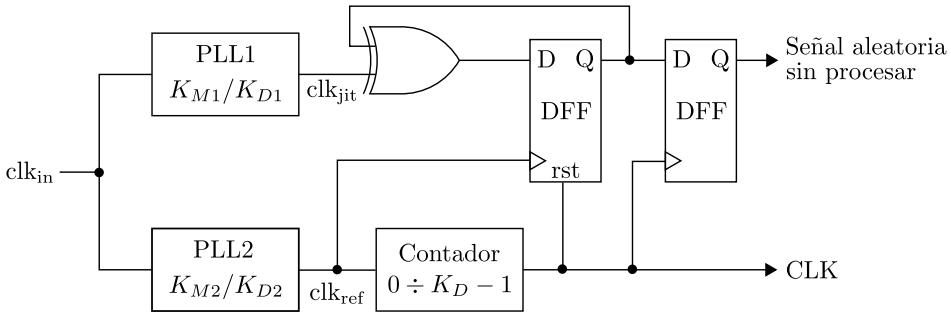


Tabla 2.1: Resumen de los resultados de implementación de las TRNGs

TRNG Type	FPGA device	Area (LUT/Reg)	Power cons. [mW]	Bit Rate [Mbits/s]	Efficiency [bits/μWs]	Entropy per bit	Entropy * Bit rate	Feasib. & Repeat.
ERO	Spartan 6	46/19	2.16	0.0042	1.94	0.999	0.004	5
	Cyclone V	34/20	3.24	0.0027	0.83	0.990	0.003	
	SmartFusion 2	45/19	4	0.014	3.5	0.980	0.013	
COSO	Spartan 6	18/3	1.22	0.54	442.6	0.999	0.539	1
	Cyclone V	13/3	0.9	1.44	1600	0.999	1.438	
	SmartFusion 2	23/3	1.94	0.328	169	0.999	0.327	
MURO	Spartan 6	521/131	54.72	2.57	46.9	0.999	2.567	4
	Cyclone V	525/130	34.93	2.2	62.9	0.999	2.197	
	SmartFusion 2	545/130	66.41	3.62	54.5	0.999	3.616	
PLL	Spartan 6	34/14	10.6	0.44	41.5	0.981	0.431	3
	Cyclone V	24/14	23	0.6	43.4	0.986	0.592	
	SmartFusion 2	30/15	19.7	0.37	18.7	0.921	0.340	
TERO	Spartan 6	39/12	3.312	0.625	188.7	0.999	0.624	1
	Cyclone V	46/12	9.36	1	106.8	0.987	0.985	
	SmartFusion 2	46/12	1.23	1	813	0.999	0.999	
STR	Spartan 6	346/256	65.9	154	2343.2	0.998	154.121	2
	Cyclone V	352/256	49.4	245	4959.1	0.999	244.755	
	SmartFusion 2	350/256	82.52	188	2286.7	0.999	188.522	

Capítulo 3

AES / algoritmo Rijndael

3.1. Evolución de los códigos y su impacto en la historia

3.2. Importancia de los códigos hoy

A medida que la información se convierte en un bien cada vez más valioso y que la revolución de las comunicaciones cambia la sociedad, el proceso de codificación de mensajes, conocido como encriptación, desempeñará un papel cada vez más importante en la vida cotidiana.

Hoy en día nuestras llamadas telefónicas rebotan en los satélites y nuestros correos electrónicos pasan por varias computadoras, y ambas formas de comunicación pueden ser interceptadas con facilidad, poniendo en peligro nuestra privacidad.

El cifrado(La encriptación) es la única forma de proteger nuestra privacidad y garantizar el éxito del mercado digital. El arte de la comunicación secreta, también conocida como criptografía, proporcionará los candados y las llaves de la era de la información.

La comunicación secreta lograda mediante la ocultación de la existencia de un mensaje se conoce como esteganografía, derivado de las palabras griegas *steganos*, que significa encubierto, y *grafo*, que significa escribir. Padece de una debilidad fundamental. Si registran al mensajero y descubren el mensaje, el contenido de la comunicación secreta se revela en el acto.

(Histaiaero afeitó la cabeza de su mensajero, escribió el mensaje en su cuero cabelludo y luego esperó a que le volviera a crecer el pelo.)

(Se escribían mensajes sobre seda fina, que luego era aplastada hasta formar una pelotita diminuta que se recubría de cera. Entonces el mensajero se trabaja la bola de cera.)

(Escribir con tinta invisible)

paralelamente al desarrollo de la esteganografía, se produjo la evolución de la crip-

tografía, término derivado de la palabra griega *kryptos*, que significa escondido. El objetivo de la criptografía no es ocultar la existencia de un mensaje, sino más bien ocultar su significado, un proceso que se conoce como codificación (Encryption). Para hacer que el mensaje sea ininteligible se codifica siguiendo un protocolo específico, sobre el cual se han puesto de acuerdo de antemano el emisor y el receptor a quien va dirigido. De esta forma, dicho receptor puede invertir el protocolo codificador y hacer que el mensaje sea comprensible. La ventaja de la criptografía es que si el enemigo intercepta un mensaje cifrado, éste es ilegible. Sin conocer el protocolo codificador, al enemigo le resultaría difícil, cuando no imposible, recrear el mensaje original a partir del texto cifrado.

De las dos ramas de la comunicación secreta, la criptografía es la más poderosa a causa de su habilidad para evitar que la información caiga en manos enemigas.

A su vez, la criptografía misma puede ser dividida en dos ramas, conocidas como **trasposición** y **sustitución**. En la trasposición, las letras del mensaje simplemente se colocan de otra manera, generando así un anagrama. Para que la trasposición sea efectiva, la combinación de letras necesita seguir un sistema sencillo, que haya sido acordado previamente por el emisor y el receptor, pero que se mantenga secreto frente al enemigo. Por ejemplo, los niños en la escuela a veces envían mensajes utilizando la trasposición de *riel*(rail fence), en la que el mensaje se escribe alternando las letras en dos líneas separadas. A continuación, la secuencia de letras de la línea inferior se añade al final de la secuencia de la línea superior, creándose así el mensaje cifrado final.

Otra forma de trasposición es la producida en el primer aparato criptográfico militar de la Historia, el escitalo espartano. El escitalo es una vara de madera sobre la que se enrosca una tira de cuero o de pergamino. El emisor escribe el mensaje a lo largo de la longitud del escitalo y luego desenrosca la tira, que ahora parece llevar una lista de letras sin sentido. El mensaje ha sido codificado. Para recuperar el mensaje, el receptor simplemente enrosca la tira de cuero en torno a un escitalo del mismo diámetro que el usado por el emisor.

La alternativa a la trasposición es la sustitución. Una de las descripciones más antiguas de codificación por sustitución aparece en el Kamasutra, un texto escrito en el siglo IV por el erudito brahmín Vatsyayana, pero que se basa en manuscritos que se remontan al siglo IV a. C. El Kamasutra recomienda que las mujeres deberían estudiar 64 artes, como cocinar, saber vestirse, dar masajes y preparar perfumes. La lista incluye también algunas artes menos obvias, como la prestidigitación, el ajedrez, la encuadernación de libros y la carpintería. El número 45 de la lista es mlecchita-vikalpa, el arte de la escritura secreta, preconizado para ayudar a las mujeres a ocultar los detalles de sus relaciones amorosas. Una de las técnicas recomendadas es emparejar al azar las letras del alfabeto y luego sustituir cada letra del mensaje original por su pareja.

Esta forma de escritura secreta se conoce como cifra de sustitución (substitution cipher) porque cada letra del texto llano (plain text) (el mensaje antes del cifrado [encryption]) se sustituye por una letra diferente para producir el texto cifrado (cyphertext) (el mensaje después del cifrado), actuando así de manera complementaria al cifrado de transposición. En la trasposición, cada letra mantiene su identidad pero cambia su posición, mientras que en la sustitución, cada letra cambia su identidad pero mantiene su posición.

Julio César utilizaba una cifra por sustitución, el simplemente reemplazó cada letra del mensaje con la letra que está tres lugares más abajo en el alfabeto. Los criptógrafos a menudo piensan en términos del alfabeto llano (plain alphabet), el alfabeto utilizado para escribir el mensaje original, y el alfabeto cifrado (cipher alphabet), las letras que sustituyen a las del alfabeto llano.

Cuando el alfabeto llano se coloca sobre el alfabeto cifrado, queda claro que el alfabeto cifrado ha sido movido tres lugares, por lo que esta forma de sustitución a menudo es llamada la cifra de cambio del César (Caesar shift cipher), o simplemente, la cifra del César. Una cifra (Cipher) es el nombre que se da a cualquier forma de sustitución criptográfica en la que cada letra es reemplazada por otra letra o símbolo.

Es evidente que al utilizar cualquier cambio de entre 1 y 25 lugares es posible generar 25 cifras distintas. De hecho, si no nos limitamos a cambiar ordenadamente el alfabeto y permitimos que el alfabeto cifrado sea cualquier combinación del alfabeto llano, podemos generar un número aún mayor de cifras distintas.

Cada una de las cifras puede ser considerada en términos de un método de codificación(encryptino) general, conocido como el algoritmo(algorithm), y una clave(key), que especifica los detalles exactos de una codificación (encryption) particular.

En este caso, el algoritmo conlleva sustituir cada letra del alfabeto llano por una letra proveniente de un alfabeto cifrado, y el alfabeto cifrado puede consistir en cualquier combinación del alfabeto llano. La clave define el alfabeto cifrado exacto que hay que usar para una codificación particular.

Un enemigo que estudie un mensaje codificado interceptado puede tener una fuerte sospecha de la existencia del algoritmo, pero quizá no conozca la clave exacta. Si el alfabeto cifrado, la clave, se mantiene como secreto bien guardado entre el emisor y el receptor, el enemigo no podrá descifrar el mensaje interceptado.

La importancia de la clave, a diferencia del algoritmo, es un principio perdurable de la criptografía. Fue expuesto definitivamente en 1883 por el lingüista holandés Augusto Kerckhoffs von Nieuwenhof en su libro La Cryptographie militaire: el Principio de Kerckhoffs: La seguridad de un cripto-sistema no debe depender de mantener secreto el cripto-algoritmo. La seguridad depende sólo de mantener secreta la clave. Además de mantener secreta la clave, un sistema de cifrado seguro debe tener también una amplia gama de claves potenciales

La cifra de sustitución monoalfabética es el nombre general que se da a cualquier cifra de sustitución en la que el alfabeto cifrado consiste en letras o en símbolos, o en una mezcla de ambos. Todas las cifras de sustitución que hemos visto hasta ahora pertenecen a esta categoría general.

Sin embargo, además de utilizar cifras, los eruditos árabes también eran capaces de destruirlas. De hecho, fueron ellos quienes inventaron el criptoanálisis, la ciencia de descifrar un mensaje sin conocer la clave. Mientras el criptógrafo desarrolla nuevos métodos de escritura secreta, es el criptoanalista el que se esfuerza por encontrar debilidades en estos métodos, para penetrar en los mensajes secretos. Los criptoanalistas árabes lograron encontrar un método para descifrar la cifra de sustitución monoalfabética, la cual había permanecido invulnerable durante muchos siglos.

El criptoanálisis no podía ser inventado hasta que una civilización hubiese alcanzado un nivel suficientemente sofisticado de erudición en varias disciplinas, incluidas las matemáticas, la estadística y la lingüística.

También analizaron las letras individuales y descubrieron en particular que algunas letras son más corrientes que otras. Las letras a y 1 son las más frecuentes en árabe, en parte a causa del artículo definido al-, mientras que letras como la j sólo aparecen con una décima parte de la frecuencia. Esta observación aparentemente inocua conduciría al primer gran avance hacia el criptoanálisis.

Al Kindi fue el autor de 290 libros de medicina, astronomía, matemáticas, lingüística y música. Su tratado más importante, que no fue redescubierto hasta 1987 en el Archivo Sulaimaniyyah Ottoman de Estambul, se titulaba Sobre el desciframiento de mensajes criptográficos

Aunque contiene detallados debates sobre estadística, fonética árabe y sintaxis árabe, el revolucionario sistema de criptoanálisis de Al Kindi está compendiado en dos breves párrafos: Una manera de resolver un mensaje cifrado, si sabemos en qué lengua está escrito, es encontrar un texto llano diferente escrito en la misma lengua y que sea lo suficientemente largo para llenar alrededor de una hoja, y luego contar cuántas veces aparece cada letra. A la letra que aparece con más frecuencia la llamamos primera, a la siguiente en frecuencia la llamamos segunda, a la siguiente tercera, y así sucesivamente, hasta que hayamos cubierto todas las letras que aparecen en la muestra de texto llano. Luego observamos el texto cifrado que queremos resolver y clasificamos sus símbolos de la misma manera. Encontramos el símbolo que aparece con más frecuencia y lo sustituimos con la forma de la letra primera de la muestra de texto llano, el siguiente símbolo más corriente lo sustituimos por la forma de la letra segunda, y el siguiente en frecuencia lo cambiamos por la forma de la letra tercera, y así sucesivamente, hasta que hayamos cubierto todos los símbolos del criptograma que queremos resolver.

La técnica de Al Kindi, conocida como análisis de frecuencia, muestra que no es necesario revisar cada una de los billones de claves potenciales.

Un código se define como una sustitución al nivel de las palabras o las frases, mientras que una cifra se define como una sustitución al nivel de las letras.

Technically, a code is defined as substitution at the level of words or phrases, whereas a cipher is defined as substitution at the level of letters. Hence the term encipher means to scramble a message using a cipher, while encode means to scramble a message using a code. Similarly, the term decipher applies to unscrambling an enciphered message, and decode to unscrambling an encoded message. The terms encrypt and decrypt are more general, and cover scrambling and unscrambling with respect to both codes and ciphers.

Técnicamente, un código se define como una sustitución al nivel de las palabras o las frases, mientras que una cifra se define como una sustitución al nivel de las letras. Por eso, el término cifrar significa ocultar un mensaje utilizando una cifra, mientras que codificar significa ocultar un mensaje utilizando un código. De manera similar, el término descifrar se aplica a la resolución de un mensaje cifrado, es decir, en cifra, y el término descodificar a la resolución de un mensaje codificado.

Los términos codificar(encrypt) y descodificar(decrypt) son más generales, y tienen relación tanto con códigos como con cifras. La Figura 7 presenta un breve resumen de estas definiciones. En general, me ajustaré a estas definiciones, pero cuando el sentido esté claro, puede que use términos como «descifrar un código» para describir un proceso que técnicamente se llamaría «descifrar una cifra» —el uso generalizado de estos términos hace que estén ampliamente aceptados.

Un nomenclador es un sistema de codificación que se basa en un alfabeto cifrado, el cual se utiliza para codificar la mayor parte de un mensaje, y en una lista limitada de palabras codificadas. Por ejemplo, un libro nomenclador podría consistir de una portada que contiene el alfabeto cifrado y una segunda página que contiene una lista de palabras codificadas. A pesar del añadido de palabras codificadas, un nomenclador no es mucho más seguro que una cifra corriente, porque la mayor parte del mensaje puede ser descifrado utilizando el análisis de frecuencia y las palabras codificadas restantes pueden ser adivinadas por el contexto.

3.3. Conceptos importantes

- Steganography - Esteganografia.
- Cryptography - Criptografía
- Encryption - Cifrado - codificación
- Transposition - Transposición
- Substitution - Sustitución

- Substitution cipher - Cifrado por sustitución
- Plaintext - texto sin formato - texto plano - texto simple - texto claro
- Ciphertext - texto cifrado
- Plain alphabet - alfabeto plano
- Cipher alphabet - alfabeto cifrado
- Caesar shift cipher - Cifrado Cesar de corrimiento
- Chipher - cifrador - codificador
- Algorithm - algoritmo
- Key - clave - llave
- Keyword - palabra clave
- Keyphrase - Frase clave
- monoalphabetic substitution cipher - Cifrado de sustitución monoalfabético
- Cryptoanalysis - criptoanálisis
- Nulls - nulos
- Code - código
- Encipher - codificar - cifrar
- Encode - codificar
- Code - código
- decipher - descifrar
- decode - descodificar
- nomenclators- nomenclador - (code book)
- Vinegere cipher
- polyalphabetic - poly alfabetico
- Charles Babbage

Capítulo 4

Pruebas NIST (National Institute of Standards and Technology)

Los generadores adecuados para su uso en aplicaciones criptográficas deben cumplir requisitos más estrictos que para otras aplicaciones. En particular, sus resultados deben ser impredecibles en ausencia de conocimiento de las entradas.

*** En este documento se discuten algunos criterios para caracterizar y seleccionar los generadores adecuados. También se trata el tema de las pruebas estadísticas y su relación con el criptoanálisis, y se ofrecen algunas pruebas estadísticas recomendadas.***

Estas pruebas pueden ser útiles como primer paso para determinar si un generador es o no adecuado para una aplicación criptográfica concreta. Sin embargo, ningún conjunto de pruebas estadísticas puede certificar de forma absoluta que un generador es adecuado para su uso en una aplicación concreta, es decir, las pruebas estadísticas no pueden sustituir al criptoanálisis.

4.1. Introducción a las pruebas de números aleatorios

La necesidad de números aleatorios y pseudoaleatorios surge en muchas aplicaciones criptográficas. Por ejemplo, los criptosistemas habituales emplean claves que deben generarse de forma aleatoria. Muchos protocolos criptográficos también requieren entradas aleatorias o pseudoaleatorias en varios puntos, por ejemplo, para las cantidades auxiliares utilizadas en la generación de firmas digitales, o para generar desafíos en los protocolos de autenticación.

Estas pruebas tratan de la comprobación de la aleatoriedad de los generadores de números aleatorios y pseudoaleatorios que pueden utilizarse para muchos fines, incluyendo aplicaciones criptográficas, de modelado y de simulación

Es un conjunto de pruebas estadísticas de aleatoriedad. El Instituto Nacional de Normas y Tecnología (NIST) considera que estos procedimientos son útiles para detec-

tar desviaciones de una secuencia binaria respecto a la aleatoriedad. Sin embargo, el probador debe tener en cuenta que las desviaciones aparentes de la aleatoriedad pueden deberse a un generador mal diseñado o a anomalías que aparecen en la secuencia binaria que se prueba (es decir, se espera un cierto número de fallos en las secuencias aleatorias producidas por un generador particular). Corresponde a la persona que realiza la prueba determinar la interpretación correcta de los resultados de la misma. *** En el apartado 4 se explica la estrategia de las pruebas y la interpretación de los resultados.***

Hay dos tipos básicos de generadores utilizados para producir secuencias aleatorias: los **generadores de números aleatorios** (RNGs) y los **generadores de números pseudoaleatorios** (PRNGs). Para las aplicaciones criptográficas, ambos tipos de generadores producen una secuencia de ceros y unos que puede dividirse en sub-secuencias o bloques de números aleatorios.

4.1.1. Aleatoriedad

Una secuencia de bits aleatoria podría interpretarse como el resultado de los lanzamientos de una moneda imparcial con caras etiquetadas como “0” y “1”, en la que cada lanzamiento tiene una probabilidad de exactamente 1/2 de producir un “0” o un “1”. Además, los lanzamientos son independientes entre sí: el resultado de cualquier lanzamiento anterior de la moneda no afecta a los lanzamientos futuros. La moneda “justa” insesgada es, por tanto, el generador perfecto de secuencias de bits aleatorias, ya que los valores “0” y “1” se distribuirán aleatoriamente (y [0,1] se distribuirá uniformemente). Todos los elementos de la secuencia se generan independientemente unos de otros, y no se puede predecir el valor del siguiente elemento de la secuencia, independientemente de cuántos elementos se hayan producido ya.

Obviamente, el uso de monedas insesgadas para fines criptográficos es poco práctico. No obstante, la salida hipotética de un generador idealizado de una secuencia aleatoria verdadera sirve como punto de referencia para la evaluación de los generadores de números aleatorios y pseudoaleatorios.

4.1.2. Imprevisibilidad

Los números aleatorios y pseudoaleatorios generados para aplicaciones criptográficas deben ser impredecibles. En el caso de los PRNGs, si la semilla es desconocida, el siguiente número de salida en la secuencia debe ser impredecible a pesar de cualquier conocimiento de los números aleatorios anteriores en la secuencia. Esta propiedad se conoce como imprevisibilidad hacia adelante. Tampoco debe ser posible determinar la semilla a partir del conocimiento de cualquier valor generado (es decir, también se requiere la imprevisibilidad hacia atrás). No debe ser evidente ninguna correlación

entre una semilla y cualquier valor generado a partir de esa semilla; cada elemento de la secuencia debe parecer el resultado de un evento aleatorio independiente cuya probabilidad es 1/2.

Para garantizar la imprevisibilidad hacia adelante, hay que tener cuidado al obtener las semillas. Los valores producidos por un PRNG son completamente predecibles si se conocen la semilla y el algoritmo de generación. Dado que en muchos casos el algoritmo de generación está disponible públicamente, la semilla debe mantenerse en secreto y no debe poder derivarse de la secuencia pseudoaleatoria que produce. Además, la propia semilla debe ser impredecible

4.1.3. Generadores de números aleatorios (RNG)

El primer tipo de generador de secuencias es un generador de números aleatorios (RNG). Un RNG utiliza una fuente no determinista (es decir, la fuente de entropía), junto con alguna función de procesamiento (es decir, el proceso de destilación de entropía) para producir aleatoriedad. El uso de un proceso de destilación es necesario para superar cualquier debilidad en la fuente de entropía que resulte en la producción de números no aleatorios (por ejemplo, la aparición de largas cadenas de ceros o unos). La fuente de entropía suele consistir en alguna cantidad física, como el ruido en un circuito eléctrico, el tiempo de los procesos del usuario (por ejemplo, las pulsaciones de las teclas o los movimientos del ratón) o los efectos cuánticos en un semiconductor. Se pueden utilizar varias combinaciones de estas entradas.

Los resultados de un RNG pueden utilizarse directamente como números aleatorios o pueden introducirse en un generador de números pseudoaleatorios (PRNG). Para ser utilizado directamente (es decir, sin procesamiento adicional), la salida de cualquier RNG necesita satisfacer estrictos criterios de aleatoriedad medidos por pruebas estadísticas para determinar que las fuentes físicas de las entradas del RNG sean aleatorias. Por ejemplo, una fuente física como el ruido electrónico puede contener una superposición de estructuras regulares, como ondas u otros fenómenos periódicos, que pueden parecer aleatorios, pero que se determinan como no aleatorios mediante pruebas estadísticas.

Para fines criptográficos, la salida de los RNGs debe ser impredecible. Sin embargo, algunas fuentes físicas (por ejemplo, los vectores fecha/hora) son bastante predecibles. Estos problemas pueden mitigarse combinando las salidas de diferentes tipos de fuentes para utilizarlas como entradas de un RNG. Sin embargo, las salidas resultantes del RNG pueden seguir siendo deficientes cuando se evalúan mediante pruebas estadísticas. Además, la producción de números aleatorios de alta calidad puede llevar demasiado tiempo, lo que hace que dicha producción no sea deseable cuando se necesita una gran cantidad de números aleatorios. Para producir grandes cantidades de números aleatorios, pueden ser preferibles los generadores de números pseudoaleatorios.

4.1.4. Generadores de números pseudoaleatorios (PRNG)

El segundo tipo de generador es un generador de números pseudoaleatorios (PRNG). Un PRNG utiliza una o más entradas y genera múltiples números "pseudoaleatorios". Las entradas de los PRNG se denominan semillas. En contextos en los que se necesita la imprevisibilidad, la propia semilla debe ser aleatoria e imprevisible. Por lo tanto, por defecto, un PRNG debe obtener sus semillas de las salidas de un RNG; es decir, un PRNG requiere un RNG como compañero.

Los resultados de un PRNG suelen ser funciones deterministas de la semilla; es decir, toda la aleatoriedad real se limita a la generación de la semilla. La naturaleza determinista del proceso da lugar al término "pseudoaleatorio". Dado que cada elemento de una secuencia pseudoaleatoria es reproducible a partir de su semilla, sólo es necesario guardar la semilla si se requiere la reproducción o validación de la secuencia pseudoaleatoria.

Irónicamente, los números pseudoaleatorios suelen parecer más aleatorios que los obtenidos de fuentes físicas. Si una secuencia pseudoaleatoria se construye correctamente, cada valor de la secuencia se produce a partir del valor anterior mediante transformaciones que parecen introducir aleatoriedad adicional. Una serie de estas transformaciones puede eliminar las autocorrelaciones estadísticas entre la entrada y la salida. Así, las salidas de un PRNG pueden tener mejores propiedades estadísticas y producirse más rápidamente que un RNG.

4.1.5. Testing (Pruebas)

Se pueden aplicar varias pruebas estadísticas a una secuencia para intentar comparar y evaluar la secuencia con una secuencia verdaderamente aleatoria. La aleatoriedad es una propiedad probabilística; es decir, las propiedades de una secuencia aleatoria pueden caracterizarse y describirse en términos de probabilidad. El resultado probable de las pruebas estadísticas, cuando se aplican a una secuencia verdaderamente aleatoria, se conoce a priori y puede describirse en términos probabilísticos. Hay un número infinito de pruebas estadísticas posibles, cada una de las cuales evalúa la presencia o ausencia de un "patrón" que, si se detecta, indicaría que la secuencia no es aleatoria. Dado que hay tantas pruebas para juzgar si una secuencia es aleatoria o no, ningún conjunto finito específico de pruebas se considera "completo". Además, los resultados de las pruebas estadísticas deben interpretarse con cierto cuidado y precaución para evitar conclusiones incorrectas sobre un generador concreto.

Una prueba estadística se formula para probar una hipótesis nula específica (**null hyphotesis**) (H0). En este caso, la hipótesis nula que se somete a prueba es que la secuencia que se comprueba es **aleatoria**. Asociada a esta hipótesis nula está la hipótesis alternativa (Ha), que, en este caso, es que la secuencia no es aleatoria. Para

cada prueba aplicada, se deriva una decisión o conclusión que acepta o rechaza la hipótesis nula, es decir, si el generador está (o no) produciendo valores aleatorios, basándose en la secuencia que se produjo.

Para cada prueba, hay que elegir un estadístico(indicador) de aleatoriedad relevante y utilizarlo para determinar la aceptación o el rechazo de la hipótesis nula. Bajo una hipótesis de aleatoriedad, dicho estadístico(indicador) tiene una distribución de valores posibles. Una distribución teórica de referencia de este estadístico(indicador) bajo la hipótesis nula se determina mediante métodos matemáticos. A partir de esta distribución de referencia, se determina un **valor crítico** (normalmente, este valor está "lejos" en las colas de la distribución, por ejemplo, en el punto del 99 %). Durante una prueba, se calcula un valor estadístico de prueba sobre los datos (la secuencia que se está probando). Este valor de la estadística de prueba se compara con el valor crítico. Si el valor de la estadística de prueba supera el valor crítico, se rechaza la hipótesis nula de aleatoriedad. En caso contrario, no se rechaza la hipótesis nula (la hipótesis de aleatoriedad) (es decir, se acepta la hipótesis).

En la práctica, la razón por la que la prueba de hipótesis estadística funciona es que la distribución de referencia y el valor crítico dependen y se generan bajo un supuesto tentativo de aleatoriedad. Si el supuesto de aleatoriedad es, de hecho, cierto para los datos en cuestión, entonces el valor estadístico de prueba calculado resultante en los datos tendrá una probabilidad muy baja (por ejemplo, 0,01 %) de superar el valor crítico. Por otro lado, si el valor calculado de la estadística de prueba supera el valor crítico (es decir, si el suceso de baja probabilidad se produce de hecho), entonces, desde el punto de vista de la prueba de hipótesis estadística, el suceso de baja probabilidad no debería producirse de forma natural. Por lo tanto, cuando el valor estadístico de la prueba calculado supera el valor crítico, se llega a la conclusión de que el supuesto original de aleatoriedad es sospechoso o defectuoso. En este caso, la prueba de hipótesis estadística arroja las siguientes conclusiones: rechazar H0 (aleatoriedad) y aceptar Ha (no aleatoriedad).

La prueba de hipótesis estadística es un procedimiento de generación de conclusiones que tiene dos resultados posibles: aceptar H0 (los datos son aleatorios) o aceptar Ha (los datos no son aleatorios). La siguiente tabla relaciona el estado verdadero (desconocido) de los datos en cuestión con la conclusión a la que se llega mediante el procedimiento de prueba.

Tabla 4.1: Tipos de errores.

Situación Verdadera	Conclusión	
	Aceptar H0	Aceptar Ha (rechazar H0)
Los datos son aleatorios (H0 es verdadera)	No error	Error de Tipo I
Los datos no son aleatorios (Ha es verdadera)	Error de Tipo II	No error

Si los datos son, en realidad, aleatorios, la conclusión de rechazar la hipótesis nula (es decir, concluir que los datos no son aleatorios) se producirá un pequeño porcentaje de veces. Esta conclusión se denomina error de Tipo I. Si los datos son, en realidad, no aleatorios, la conclusión de aceptar la hipótesis nula (es decir, concluir que los datos son realmente aleatorios) se denomina error de Tipo II. Las conclusiones de aceptar H0 cuando los datos son realmente aleatorios, y de rechazar H0 cuando los datos son no aleatorios, son correctas.

La probabilidad de un error de Tipo I suele denominarse **nivel de significación** de la prueba. Esta probabilidad puede establecerse antes de una prueba y se denota como α. Para la prueba, α es la probabilidad de que la prueba indique que la secuencia no es aleatoria cuando realmente lo es. Es decir, una secuencia parece tener propiedades no aleatorias incluso cuando un "buen" generador produjo la secuencia. Los valores habituales de α en criptografía son de aproximadamente 0,01.

La probabilidad de un error de Tipo II se denota como β. Para la prueba, β es la probabilidad de que la prueba indique que la secuencia es aleatoria cuando no lo es; es decir, un generador "malo" produjo una secuencia que parece tener propiedades aleatorias. A diferencia de α, β no es un valor fijo. β puede tomar muchos valores diferentes porque hay un número infinito de formas en que una secuencia de datos puede ser no aleatoria, y cada forma diferente produce un β diferente. El cálculo del error de Tipo II β es más difícil que el cálculo de α debido a los muchos tipos posibles de no aleatoriedad.

Uno de los principales objetivos de las siguientes pruebas es minimizar la probabilidad de un error de Tipo II, es decir, minimizar la probabilidad de aceptar una secuencia producida por un generador como buena cuando el generador era realmente malo. Las probabilidades α y β están relacionadas entre sí y con el tamaño n de la secuencia probada, de manera que si se especifican dos de ellas, el tercer valor se determina automáticamente. Los profesionales suelen seleccionar un tamaño de muestra n y un valor para α (la probabilidad de un error de Tipo I, el nivel de significación). A continuación, se selecciona un punto crítico para una estadística determinada que produzca el menor β (la probabilidad de un error de Tipo II). Es decir, se selecciona un tamaño de muestra adecuado junto con una probabilidad aceptable de decidir que un mal generador ha producido la secuencia cuando realmente es aleatoria. A continuación, se elige el punto de corte de la aceptabilidad de forma que la probabilidad de aceptar falsamente una secuencia como aleatoria tenga el menor valor posible.

Cada prueba se basa en un valor estadístico de prueba calculado, que es una función de los datos. Si el valor estadístico de la prueba es S y el valor crítico es t, entonces la probabilidad de error de Tipo I es :

$$P(S > t||\text{Ho es verdadera}) = P(\text{rechazar Ho}|\text{H0 es verdadera}) \tag{4.1}$$

, y la probabilidad de error de Tipo II es

$$P(S \leq t|\text{H0 es falsa}) = P(\text{aceptar H0}|\text{H0 es falsa}) \tag{4.2}$$

La estadística de la prueba se utiliza para calcular un P-value que resume la fuerza de la evidencia contra la hipótesis nula. Para estas pruebas, cada P-value es la probabilidad de que un generador de números aleatorios perfecto hubiera producido una secuencia menos aleatoria que la secuencia que se probó, dado el tipo de no aleatoriedad evaluado por la prueba. Si el P-value de una prueba es igual a 1, la secuencia parece tener una aleatoriedad perfecta. Un P-value de cero indica que la secuencia parece ser completamente no aleatoria. Se puede elegir un nivel de significación (α) para las pruebas. Si el valor $P \geq \alpha$, se acepta la hipótesis nula; es decir, la secuencia parece ser aleatoria. Si el valor $P < \alpha$, entonces se rechaza la hipótesis nula; es decir, la secuencia parece no ser aleatoria. El parámetro α denota la probabilidad del error de Tipo I. Normalmente, α se elige en el rango [0,001, 0,01].

Capítulo 5

Conclusiones

5.1. Fichas bibliográficas

1. 2021 - Luis Gerardo De La Fraga - Designing an authenticated Hash function with a 2D chaotic map. [1]

2. 2017 - Luis Gerardo De La Fraga - Hardware implementation of pseudo-random number generators based on chaotic maps. [2]

3. 2021 - Sundarapandian Vaidyanathan - A 5-D Multi-Stable Hyperchaotic Two-Disk Dynamo System With No Equilibrium Point: Circuit Design, FPGA Realization and Applications to TRNGs and Image Encryption. [3]

4. 2020 - N. Nalla Anandakumar - FPGA-Based True Random Number Generation Using Programmable Delays in Oscillator-Rings. [4]

5. 1999 - Benjamin Jun and Paul Kocher - The Intel Random Number Generator. [5]

6. 2021 - Xinyu Wang - High-Throughput Portable True Random Number Generator Based on Jitter-Latch Structure. [6]

7. 2021 - Adriaan Peetermans - Design and Analysis of Configurable Ring Oscillators for True Random Number Generation Based on Coherent Sampling. [7]

8. 2016 - Oto Petura - A Survey of AIS-20/31 Compliant TRNG Cores Suitable for FPGA Devices. [8]

9. 1993 - J.C. Sprott - Automatic generation of strange attractors. [9]

10. 2015 - Patrick Haddad - A Physical Approach for Stochastic Modeling of TERO-Based TRNG. [10]

11. 2010 - Mathieu Baudet - On the Security of Oscillator-Based Random Number Generators. [11]
12. 2010 - Michal Varchola - New High Entropy Element for FPGA Based True Random Number Generators. [12]
13. 2019 - Viktor Fischer - Modern random number generator design – Case study on a secured PLL-based TRNG [13]
14. 2021 - Soyeon Choi - Analysis of Ring Oscillator based True Random Number Generator on FPGAs [14]
15. 2021 - Claudio Garcia - Chaotic Features of a Class of Discrete Maps without Fixed Points [15]
16. 2020 - Rethinam Sivaraman - Ring oscillator as confusion diffusion agent: a complete TRNG drove image security [16]
17. 2020 - Joan Daemen - The design of Rijndael: AES The Advanced Encryptation Standard.[17]
18. 2003 - Viktor Fischer - True Random Number Generator Embedded in Reconfigurable Hardware [18]
19. 2018 - Viktor Fischer - From Physical to Stochastic Modeling of a TERO-Based TRNG [19]
20. 2011 - Viktor Fischer - Security Trends for FPGAS [20]
21. 2016 - Viktor Fischer - Platform for Testing and Evaluation of PUF and TRNG Implementations in FPGAs [21]

Apéndice A

Códigos

```
clear; close all; clc;
iter = 100000;
n = 0:1:iter;
a = [-0.6, -0.1, 1.1, 0.2, -0.8, 0.6, -0.7, 0.7, 0.7, 0.3, 0.6, 0.9];        %M1
x0 = 0.1; y0 = 0.2;
xn = zeros(1,iter+1); yn = zeros(1,iter+1);
xn(1) = x0; yn(1) = y0;
for i = 2:iter+1
    xn(i) = a(1) + a(2)*xn(i-1) + a(3)*xn(i-1)^(2) + a(4)*xn(i-1)*yn(i-1) + a(5)*yn(i-1) +
     a(6)*yn(i-1)^(2);
    yn(i) = a(7) + a(8)*xn(i-1) + a(9)*xn(i-1)^(2) + a(10)*xn(i-1)*yn(i-1) + a(11)*yn(i-1)
     + a(12)*yn(i-1)^(2);
end
plot(xn,yn,'.k','MarkerSize',0.5); grid on; xlabel("xn"); ylabel("yn"); axis equal;
```

Código A.1: Nombre de abajo

```
library ieee;
use ieee.std_logic_1164.all;

entity mux is
  generic( n : integer := 64);  -- Tamanio de palabra
  port(
        X0  : in std_logic_vector(n-1 downto 0);
        Xn_1: in std_logic_vector(n-1 downto 0);
        SEL : in std_logic;
        Xn  : out std_logic_vector(n-1 downto 0)
  );
end;

architecture arch of mux is
begin
  Xn <= X0 when SEL = '0' else Xn_1;
end arch;
```

Código A.2: Nombre de abajo

Bibliografía

[1] L. G. D. la Fraga, C. Mancillas-López, and E. Tlelo-Cuautle, "Designing an authenticated hash function with a 2d chaotic map," *Nonlinear Dynamics*, vol. 104, pp. 4569–4580, may 2021.

[2] L. G. de la Fraga, E. Torres-Pérez, E. Tlelo-Cuautle, and C. Mancillas-López, "Hardware implementation of pseudo-random number generators based on chaotic maps," *Nonlinear Dynamics*, vol. 90, pp. 1661–1670, aug 2017.

[3] S. Vaidyanathan, A. Sambas, B. Abd-El-Atty, A. A. A. El-Latif, E. Tlelo-Cuautle, O. Guillen-Fernandez, M. Mamat, M. A. Mohamed, M. Alcin, M. Tuna, I. Pehlivan, I. Koyuncu, and M. A. H. Ibrahim, "A 5-d multi-stable hyperchaotic two-disk dynamo system with no equilibrium point: Circuit design, FPGA realization and applications to TRNGs and image encryption," *IEEE Access*, vol. 9, pp. 81352–81369, 2021.

[4] N. N. Anandakumar, S. K. Sanadhya, and M. S. Hashmi, "FPGA-based true random number generation using programmable delays in oscillator-rings," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 67, pp. 570–574, mar 2020.

[5] B. Jun and P. Kocher, "The intel random number generator," *Cryptography Res. Inc., San Francisco, CA, USA*, 1999.

[6] X. Wang, H. Liang, Y. Wang, L. Yao, Y. Guo, M. Yi, Z. Huang, H. Qi, and Y. Lu, "High-throughput portable true random number generator based on jitter-latch structure," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 68, pp. 741–750, feb 2021.

[7] A. Peetermans, V. Rožić, and I. Verbauwhede, "Design and analysis of configurable ring oscillators for true random number generation based on coherent sampling," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 14, pp. 1–20, jun 2021.

[8] O. Petura, U. Mureddu, N. Bochard, V. Fischer, and L. Bossuet, "A survey of AIS-20/31 compliant TRNG cores suitable for FPGA devices," in *2016 26th International Conference on Field Programmable Logic and Applications (FPL)*, IEEE, aug 2016.

[9] J. Sprott, "Automatic generation of strange attractors," *Computers & Graphics*, vol. 17, pp. 325–332, may 1993.

[10] P. Haddad, V. Fischer, F. Bernard, and J. Nicolai, "A physical approach for stochastic modeling of TERO-based TRNG," in *Lecture Notes in Computer Science*, pp. 357–372, Springer Berlin Heidelberg, 2015.

[11] M. Baudet, D. Lubicz, J. Micolod, and A. Tassiaux, "On the security of oscillator-based random number generators," vol. 24, pp. 398–425, oct 2010.

[12] M. Varchola and M. Drutarovsky, "New high entropy element for FPGA based true random number generators," pp. 351–365, Springer Berlin Heidelberg, 2010.

[13] V. Fischer, F. Bernard, and N. Bochard, "Modern random number generator design – case study on a secured PLL-based TRNG," *it - Information Technology*, vol. 61, pp. 3–13, jan 2019.

[14] S. Choi, Y. Shin, and H. Yoo, "Analysis of ring-oscillator-based true random number generator on FPGAs," in *2021 International Conference on Electronics, Information, and Communication (ICEIC)*, IEEE, jan 2021.

[15] C. García-Grimaldo and E. Campos, "Chaotic features of a class of discrete maps without fixed points," *International Journal of Bifurcation and Chaos*, vol. 31, oct 2021.

[16] R. Sivaraman, S. Rajagopalan, J. B. B. Rayappan, and R. Amirtharajan, "Ring oscillator as confusion – diffusion agent: a complete TRNG drove image security," *IET Image Processing*, vol. 14, pp. 2987–2997, oct 2020.

[17] V. R. Joan Daemen, *The Design of Rijndael.* Springer-Verlag GmbH, May 2020.

[18] V. Fischer and M. Drutarovský, "True random number generator embedded in reconfigurable hardware," in *Cryptographic Hardware and Embedded Systems - CHES 2002*, pp. 415–430, Springer Berlin Heidelberg, 2003.

[19] F. Bernard, P. Haddad, V. Fischer, and J. Nicolai, "From physical to stochastic modeling of a TERO-based TRNG," *Journal of Cryptology*, vol. 32, pp. 435–458, mar 2018.

[20] B. Badrignans, J. L. Danger, V. Fischer, G. Gogniat, and L. Torres, eds., *Security Trends for FPGAS*. Springer Netherlands, 2011.

[21] M. Laban, Milos Drutarovsky, V. Fischer, and M. Varchola, "Platform for testing and evaluation of puf and trng implementations in fpgas," 2016.